



MEMORY MANAGEMENT

Memory Management- Need???

- To **improve the utilization of the CPU** and the speed of the computer's **response** to its users, the system keeps several processes in memory, that is, several **processes share the memory**.
- Due to the sharing of memory, there is **need for memory management**.
- It is the job of **memory manager** to manage memory between multiple processes in an efficient way.
- Keeps track of **which part of the memory is occupied** and which part is free.
- **Allocates and de-allocates** memory to the processes whenever required.
- **Provides the protection mechanism** to protect the memory allocated to each process from being accessed by other processes.
- For managing the memory- a number of available strategies.
- Memory allocation to the processes using these strategies is either ***contiguous or non- contiguous***.

Physical Address

- Every byte in the memory has a specific address that may range from 0 to some maximum value as defined by the hardware.
- This address is known as **physical address**.
- Physical address is the actual location in the **main memory (RAM)** where data or instructions are stored.
- It is the address that the **memory hardware (RAM) understands**.
- Whenever a program is brought into main memory for execution, it occupies certain number of memory locations.
- The set of all physical addresses used by the program is known as **physical address space**.

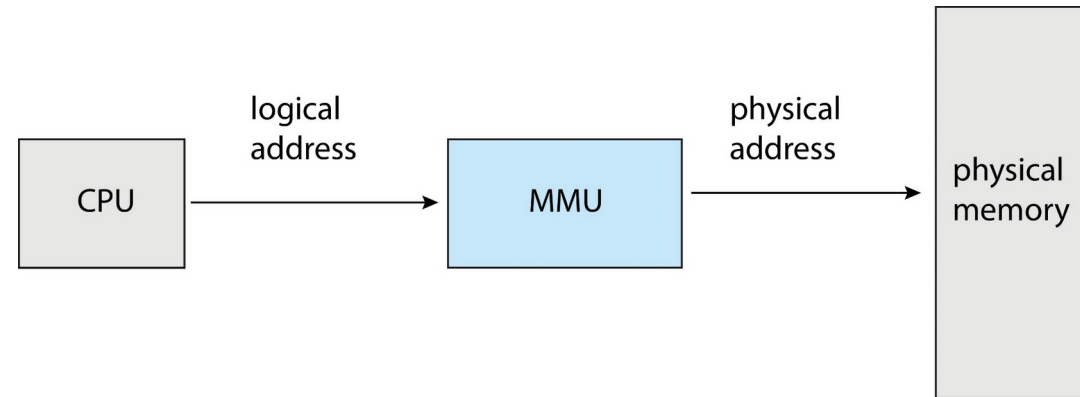
Logical Address

- A **logical address** (also called a **virtual address**) is the address **generated by the CPU** while a program is executing.
- It is the address that the **program uses to access its instructions and data**.
- The logical address is **not the actual address in RAM**. Before the data can be accessed, the **Memory Management Unit (MMU)** translates the logical address into a **physical address**.
- Before a program can be executed, it must be compiled to produce the **machine code**.
- Accordingly all the variables and procedures used in the source program are assigned some specific address known as **logical address**.
- In machine code, all **references to data or code are made by specifying the logical addresses** and not by the variable or procedure names.
- The set of all logical addresses used by a user program is known as its **logical address space**.



Memory-Management Unit (MMU)

- Hardware device that at runtime maps virtual to physical address



- Many methods possible.



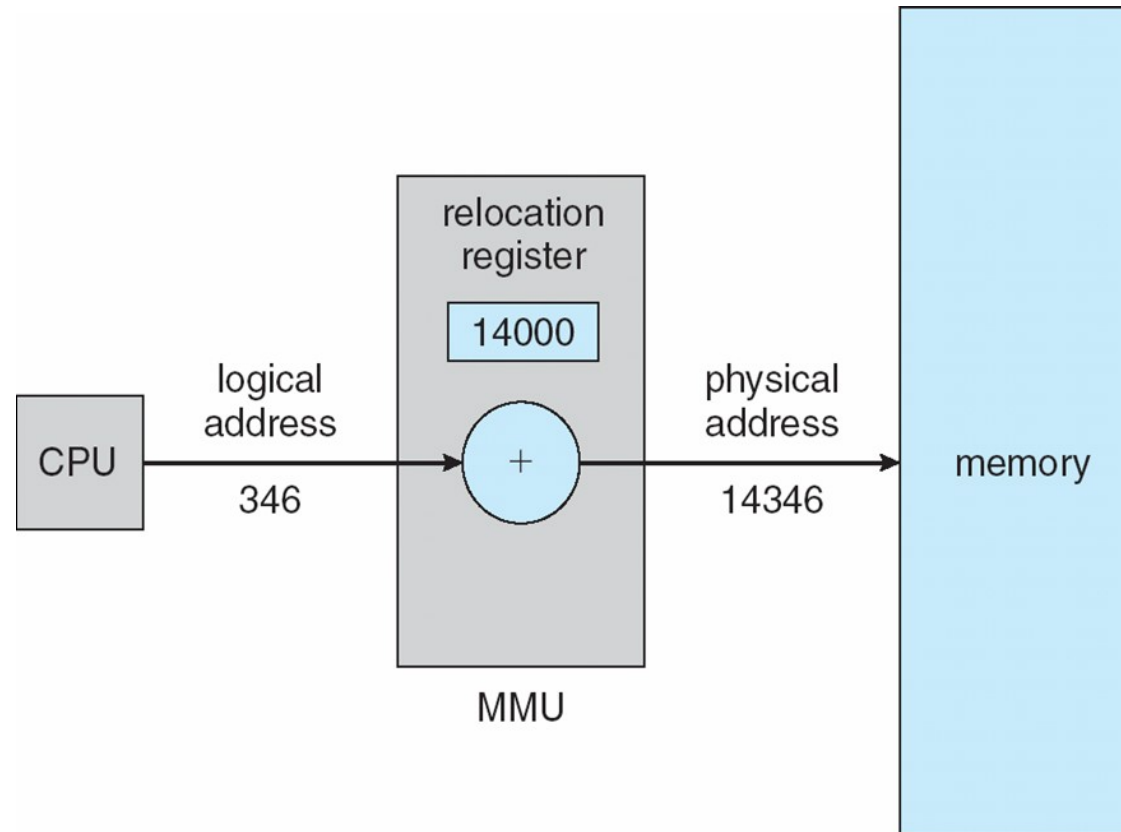
Memory Management Unit(MMU)

- When a user program is brought into main memory for execution, its logical addresses must be mapped to physical addresses.
- The mapping from addresses associated with a program to memory addresses is known as **address binding**.
- The address binding can take place at one of the following times:
 - **Compile time:** The address binding takes place at compile time if it is known at the compile time which addresses the program will occupy in the main memory. In this case, the program generates **absolute code** at the compile time only, that is, logical addresses are same as that of physical addresses.
 - **Load time:** The address binding occurs at load time if it is not known at the compile time which addresses the program will occupy in the main memory. In this case, the program generates **relocatable code** at the compile time which is then converted into the absolute code at the load time.
 - **Run time:** The address binding occurs at run time if the process is supposed to move from one memory segment to other during its execution. In this case also, the program generates relocatable code at compile time which is then converted into the absolute code at the run time.

Note: The run time address binding is performed by the hardware device known as **memory-management unit (MMU)**.

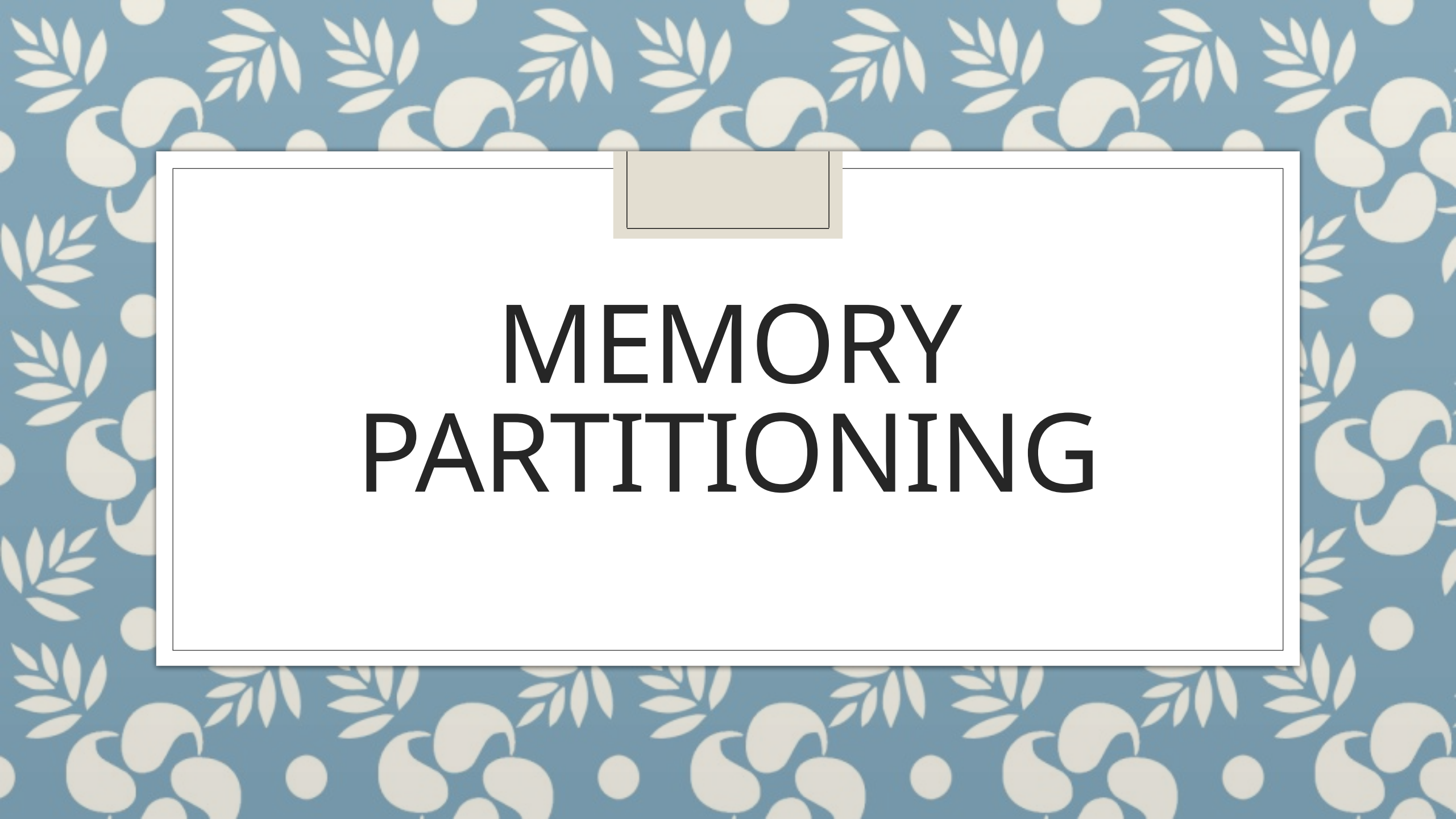


Dynamic relocation using a relocation register



Relocation Register is also known as **Base Register**.





MEMORY PARTITIONING

Bare Machine

- A **bare machine** refers to a computer having no operating system.
- In the early stage of computing, there were no operating systems.
- Programmers used to feed their programs into the computer system directly using machine language—no system software support was there- **bare machine approach** in the development of operating systems.
- The **whole memory is assigned to the user process** which runs in the kernel mode.
- The user process has **complete control over the memory**.
- **No protection mechanisms** are required as no operating system or any other process is residing in the memory. Thus, there are **no overheads on the programmers** to provide security and protection mechanisms.
- In addition, **no special memory hardware is needed**, which reduces the hardware cost to a great extent.
- Moreover, the **logical address** generated by the user programs **equal to the physical address** of the memory, hence **no address translation is required**. Therefore, **no memory management strategies** are required in a bare machine.
- However, since there are no **operating system services** for accessing the main memory, the **user must provide these**.

Memory Management Strategies

- **Contiguous Memory Management Strategies**

- *Single Partition*
 - *Multiple Partitions.*
 - **Fixed sized(equal- sized) partitions**
 - **Variable sized(unequal-sized) partitions**

- **Non- Contiguous Memory Management Strategies**

- Paging
 - Demand Paging
 - Segmentation

Contiguous Partitioning

- In contiguous memory allocation, each process is allocated a single contiguous part of the memory.
- Memory management schemes that are based on this approach are
 - *Single Partition*
 - *Multiple Partitions.*
 - Fixed sized(equal- sized) partitions
 - Variable sized(unequal-sized) partitions

Single Partition

- Simplest way- partition the main memory into **two parts**.
- **One** of them is permanently allocated to the **operating system** while the **other** part is allocated to the **user process**
- In order to provide a contiguous area for the user process, **OS usually resides at one extreme end of memory**.
- The operating system is in lower part of the memory. However, it is not essential that the operating system resides at the bottom of memory; it can reside at the upper part of the memory also.
- The factor that decides the location of the operating system in the memory is the location **of the interrupt vector**.
- **The operating system is placed at the same end of the memory where the interrupt vector is located.**

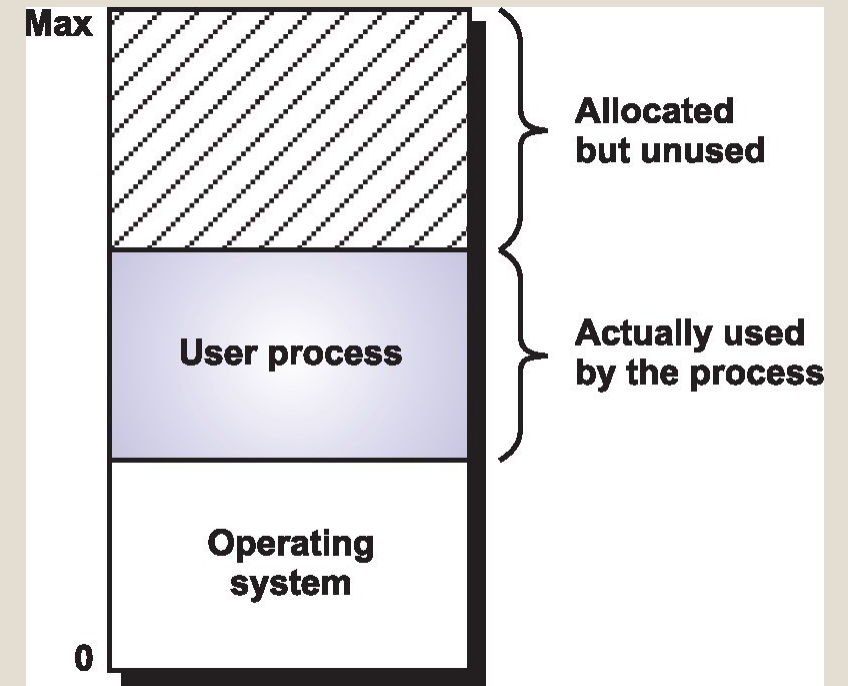


Fig. 7.1 Memory having Single Partition

Note: The portion of the operating system residing in the memory is known as **resident monitor**.

Hardware Support

- Single contiguous memory allocation is a simple memory management scheme that is **usually associated with stand-alone computers** with simple batch operating systems.
- No special hardware support is needed, except for **protecting the operating system** from the user process.
- This **hardware protection mechanism** may include **bounds register** and **two modes of CPU** (supervisor and user).
- The **address of protected area** is contained in **bounds register**.
- Depending upon the mode of CPU, **access to protected area** is controlled.
- If CPU is in **user mode**, each time a memory reference is made, **a hardware check is performed to ensure that it is not an access to the protected area**.
- If an attempt is made to access the protected area, **an interrupt is generated** and **control is transferred** to the operating system.
- On the other hand, if the CPU is in **supervisor mode**, the operating system is allowed to **access** the protected area **as well as to execute the privileged instructions to alter the content** of bounds register.

Software Support

- Only **one process** can execute **at a time**.
- Whenever a process is to be executed, **the size of the process is checked**.
- If the **size is less or equal to size of memory**, the operating system loads it into the main memory for execution.
- **After termination** of that process, the operating system waits for **another process**.
- If the **size** of the process is **greater than the size of memory**, an **error occurs and next process is scheduled**.
- The same sequence is performed for the next process.

Advantages

- **Easy** to implement.
- Generally, the operating system needs **to keep track of the first and the last location** allocated to the user processes.
- In this case, **the first location** is immediately following the operating system and the **last location** is determined by the **capacity of the memory**.
- It needs **no hardware support except for protecting** the operating system from the user process

Disadvantages

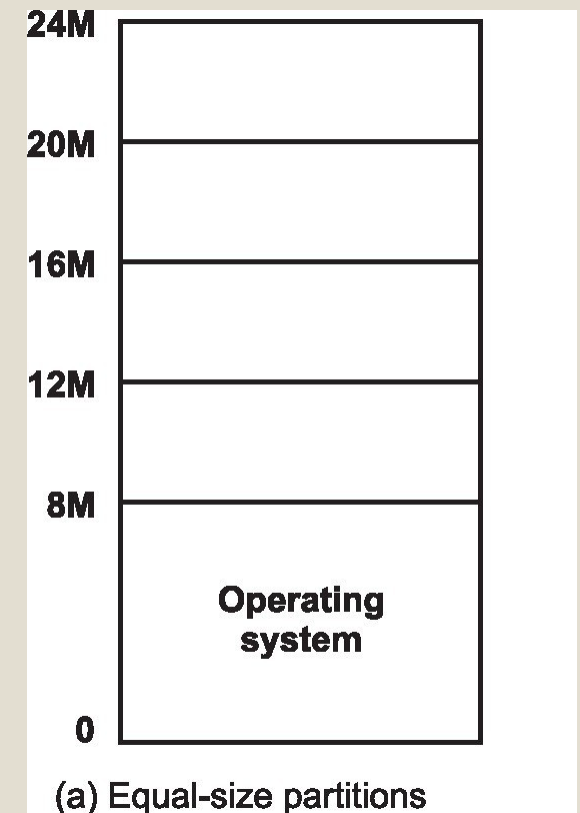
1. Only **one user process can execute at a time**, the portion of the **memory** which is allocated but not used by the process will get **wasted**.
 - Thus, **memory is not fully utilized**.
2. The **process size must be smaller or equal to the size of main memory**; otherwise, the process cannot be executed.

Multiple Partitions

- Allows multiple processes to reside in the memory at the same time.
- The simple way to achieve multiprogramming is to divide main memory into a number of partitions which may be of fixed or variable size.

Multiprogramming with Fixed Partitions

- **Fixed-sized partitions** - any process can be loaded into any partition.
- Whenever a **partition is free, a process whose size is less than or equal to the partition size** is selected from the input queue and loaded into this partition.
- When the **process terminates**, the **partition becomes free** to be allocated to another process.
- The **implementation** of this method **does not require much effort** since all partitions are of same size.
- The operating system is **required to keep track only of the partition occupied by each process**.
- For this, it **maintains a table that keeps either the starting address of each process or the partition number** occupied by each process.



Adv. & Disadv.

Advantages

- The fixed-partitioning technique is **easy to implement** and **requires less overhead**.

Drawbacks

- The **number of processes in memory depends on the number of partitions**. Thus, the **degree of multiprogramming is limited**.
 - The memory **cannot be used efficiently in the case of processes of small sizes**.
 - **Internal fragmentation**
- **Note:** *The technique having fixed partitions is no longer in use.*

Major Drawback

- There is one problem with this method that is the **memory utilization is not efficient.**
- Any **process** regardless of how **small** it is, **occupies an entire partition** which leads to the **wastage of memory** within the partition.
- This phenomenon which results in the wastage of memory within the partition is called **internal fragmentation.**
- For example, loading a process of size $4M - n$ bytes into a partition of size $4M$ (where, M stands for megabytes) would result in a wasted space of n bytes within the partition

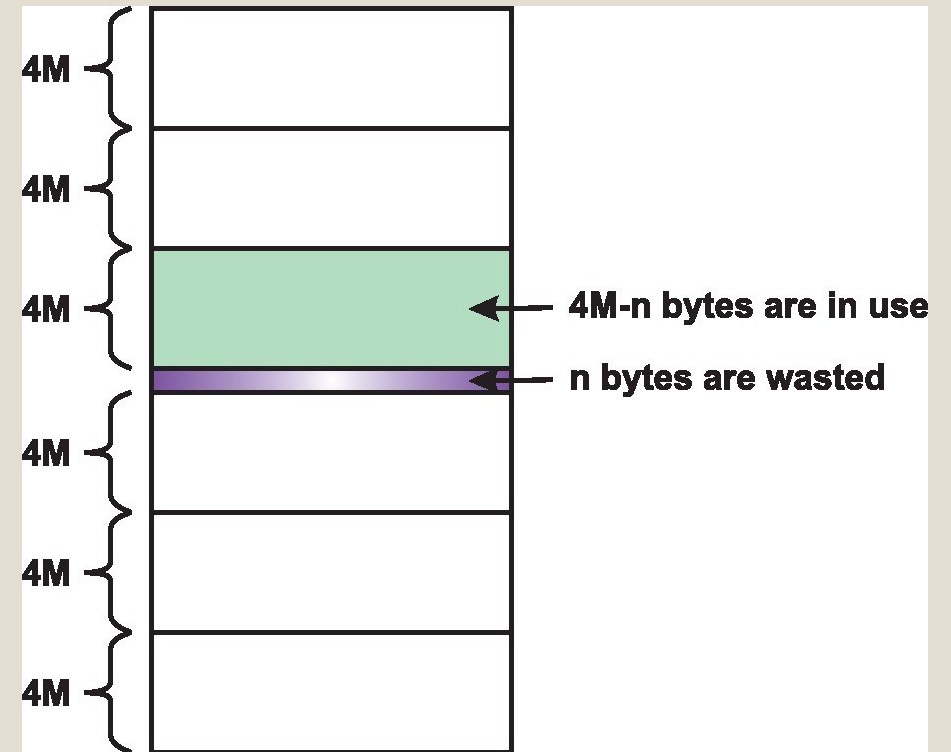
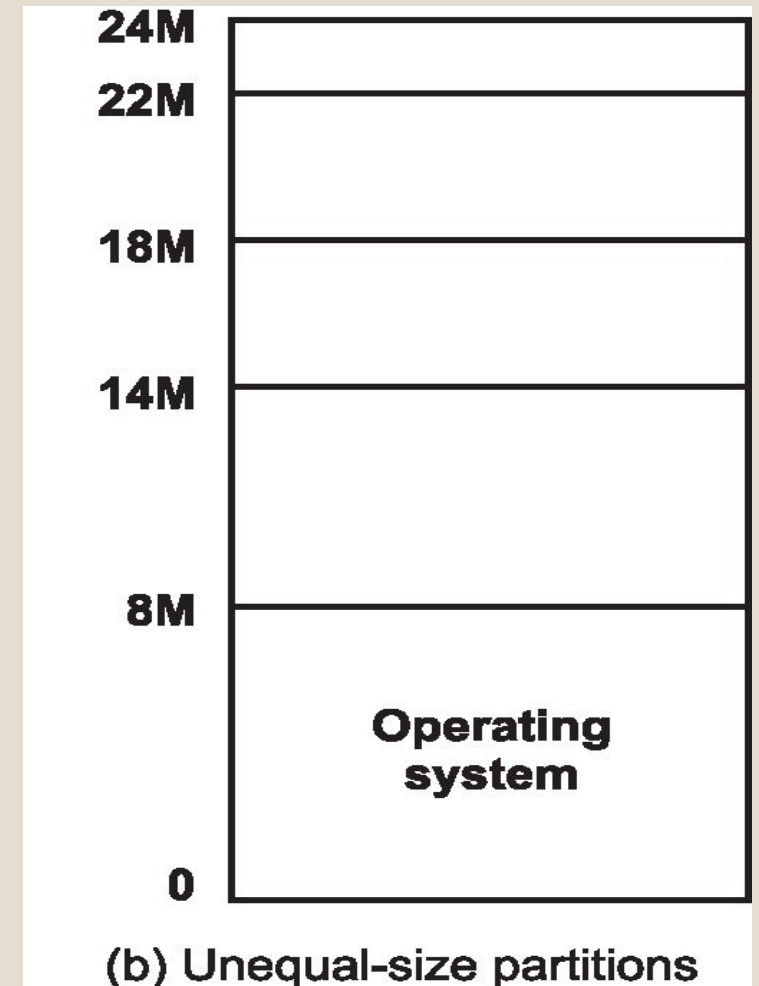


Fig. 7.3 Internal Fragmentation

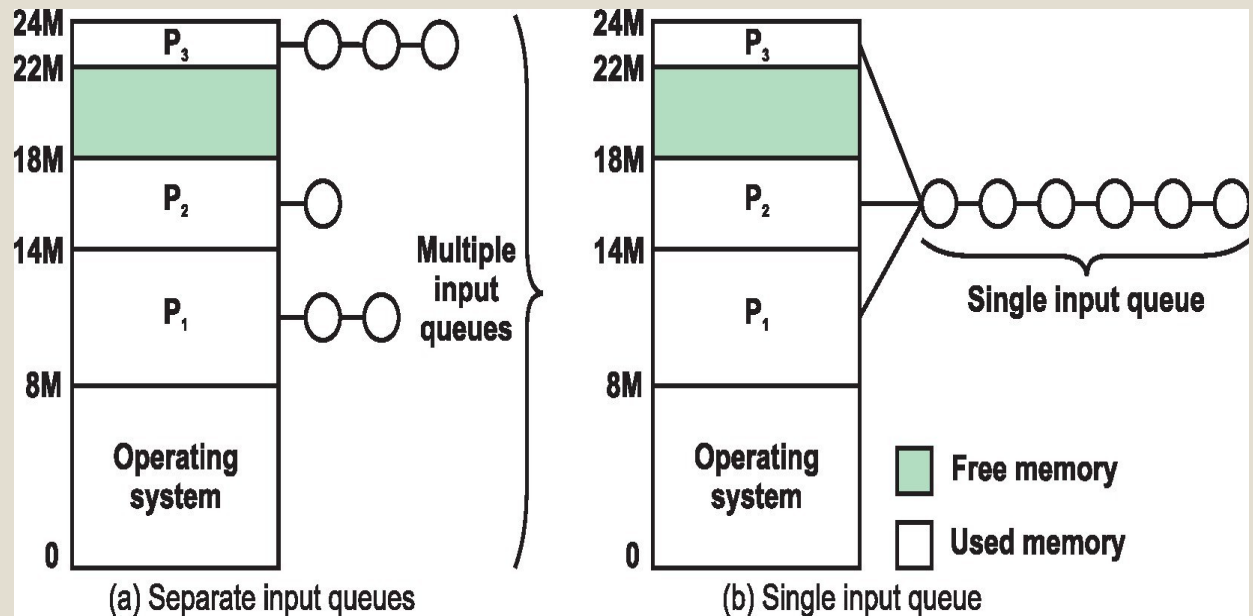
Solution

- Cannot be resolved completely, but **can be reduced** to some extent.
- Use **variable sized partition** method where **a separate input queue is maintained for each partition**.
- Whenever a process arrives, **it is placed into the input queue of the smallest partition large enough to hold it**.
- When this **partition becomes free**, it is **allocated** to the process.
- For example, if a process of size 5M arrives, it will be accommodated in the partition of size 6M.
- **In this case also, some memory is wasted**, that is, internal fragmentation still exists, **but less than that of equal-sized partition method**.



Challenge & Solution

- **Problem-** the input queue for a large partition is empty but the queue for small partition is full.
- That is, the small jobs have to wait to be loaded into memory, though a large amount of memory is free.
- **Solution-** a single input queue can be maintained.
- Whenever a partition becomes free, the process that fits in it can be chosen from the input queue using some scheduling algorithm.



Multiprogramming with Variable Partitions-

A different approach

- Initially, when there is **no process in the memory**, the **whole memory is available** for the allocation.
- It is considered as **a single large partition of available memory** (a **hole**).
- Whenever a process **requests** for the memory, **the hole large enough to accommodate that process is allocated**.
- The **rest** of the memory is **available to other processes**.
- As soon as the **process terminates**, the **memory** occupied by it is **de-allocated** and can be used for other processes.
-

Multiprogramming with Variable Partitions

- Due to subsequent memory allocations and de-allocations, at a given time, **some parts of memory will be in use while others will be free.**
- Now to make further allocations, **the memory manager must keep track of the free space in memory.**
- For this, the memory manager **maintains a free-storage list that keeps track of the unused part (holes of variable sizes) of memory.**
- The free-storage list is **implemented as a linked list where each node contains the size of the hole and the address of the next available hole.**

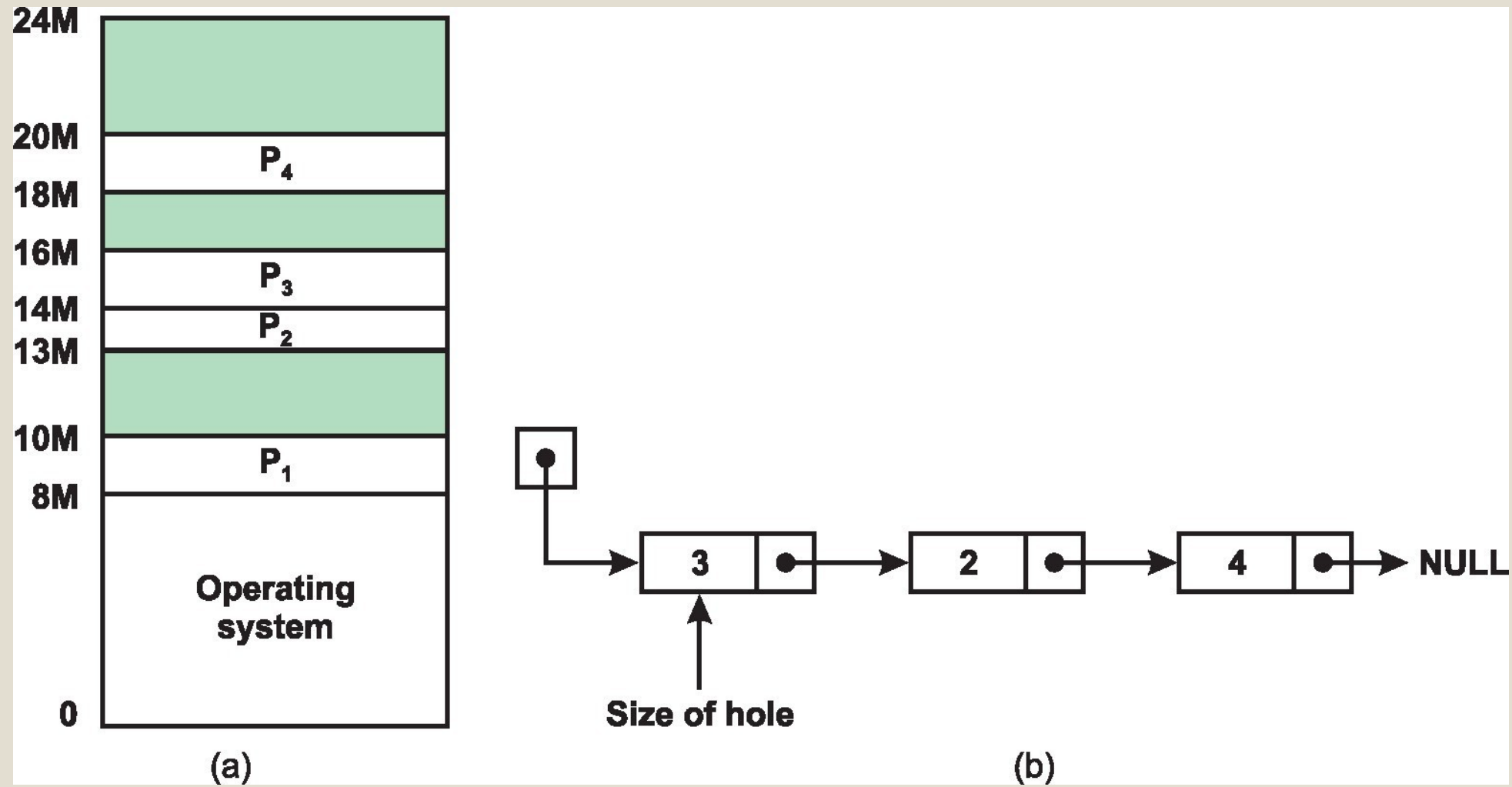


Fig. 7.5 Memory Map and Free-storage List

Multiprogramming with Variable Partitions

- In general, at a certain point of time, there will be a set of holes of various sizes dispersed in the memory.
- As a result, there may be possibility that the total available memory is large enough to accommodate the waiting process.
- However, it cannot be utilized as it is scattered. This results into external fragmentation problem

Partition Selection Algorithms

◦ Whenever a process arrives and there are various holes large enough to accommodate it, the operating system may use one of the following algorithms to select a partition for the process.

- **First fit:**
 - In this algorithm, the operating system scans the free- storage list.
 - Allocates the first hole that is large enough to accommodate that process.
 - This algorithm is **fast** because search is little as compared to other algorithms.
- **Best fit:**
 - In this algorithm, the operating system scans the free- storage list.
 - Allocates the smallest hole whose size is larger than or equal to the size of the process - it allocates a partition that is close to the size required for that process.
 - It is **slower** than the first fit algorithm as it has to search the entire list every time.
 - Moreover, it leads to more wastage of memory as it results in the smallest leftover holes that cannot be used to satisfy the memory allocation request.

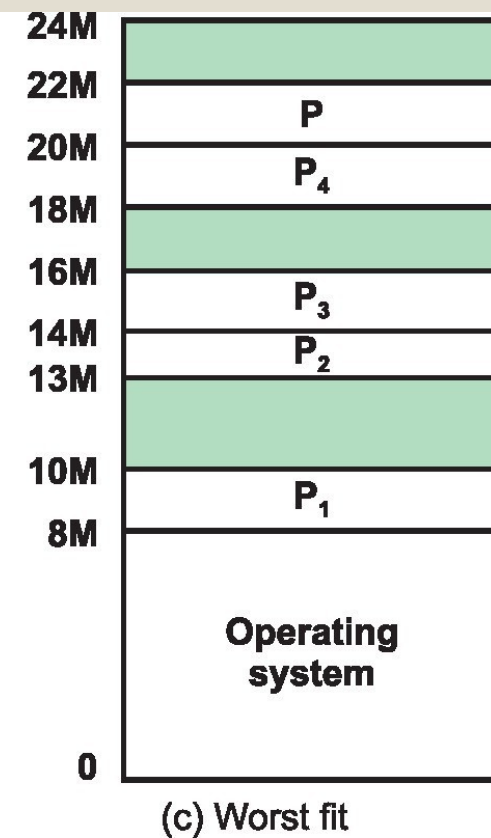
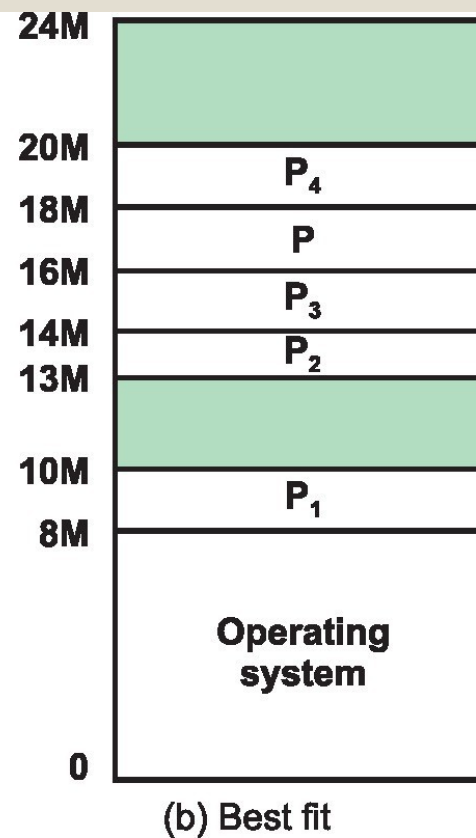
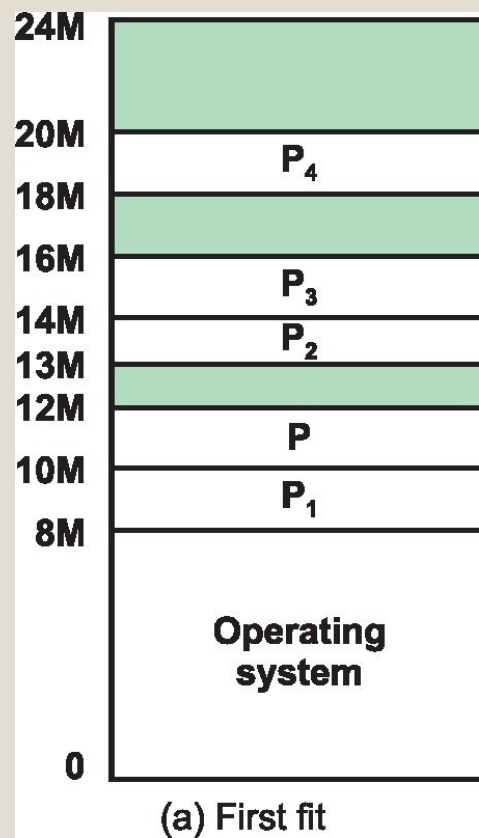
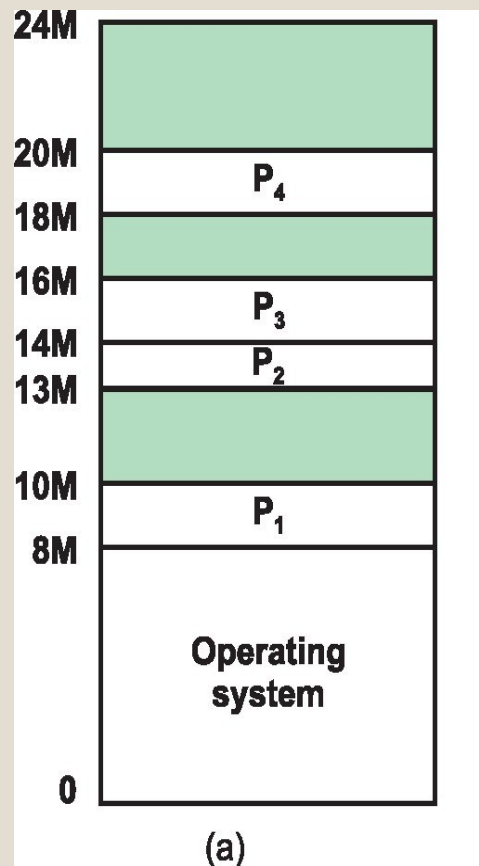
Partition Selection Algorithms

- **Worst fit:**
- In this algorithm, the operating system scans the entire free-storage list.
- **Allocates the largest hole** to that process.
- Unlike best fit, this algorithm **results in the largest leftover holes**.
- Moreover, simulation indicates that **worst fit allocation is not very effective in reducing the wastage of memory**.

- **Next fit:**
- This algorithm to first fit, but with a key difference.
- Instead of always starting the search for an available memory block from the beginning of the free memory list, next fit begins searching from where the previous allocation left off.
- This **helps avoid repeatedly allocating from the same initial blocks and can lead to better memory utilization**.

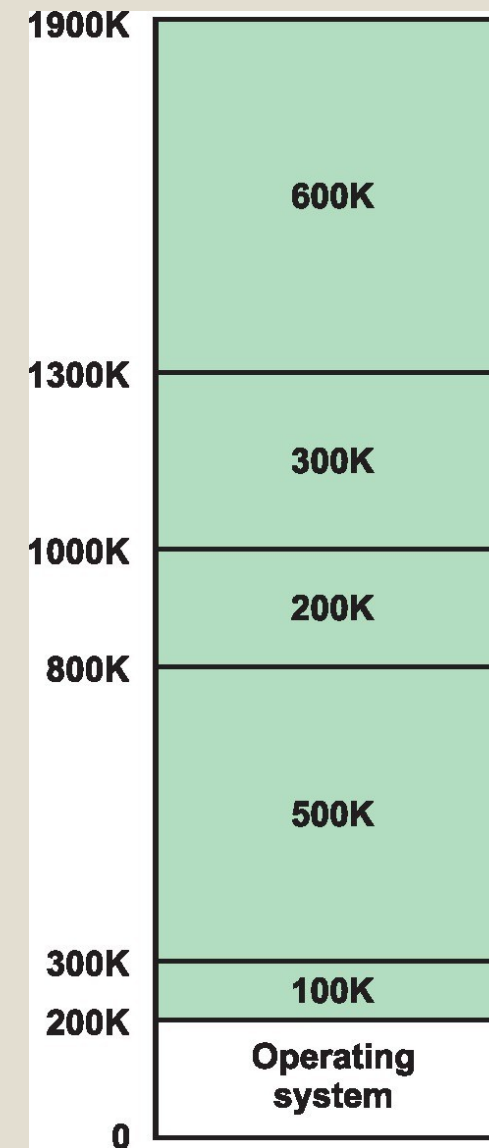
Example 1: Consider the memory map given in Figure. how would each of the first fit, best fit and worst fit algorithms allocate memory to a process P of size 2M.

Solution: According to the different algorithms, memory will be allocated to the process P

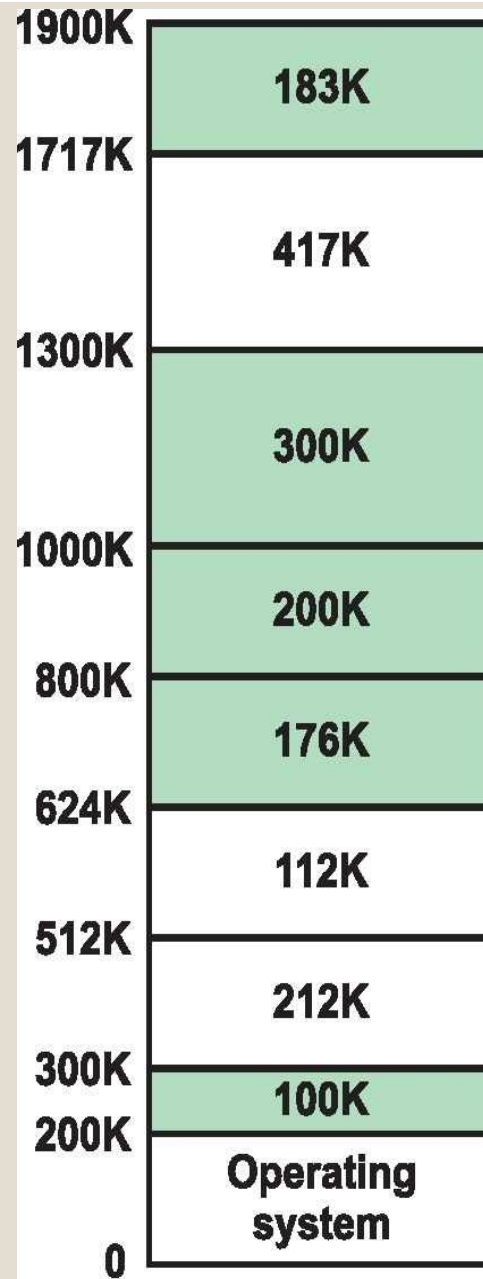
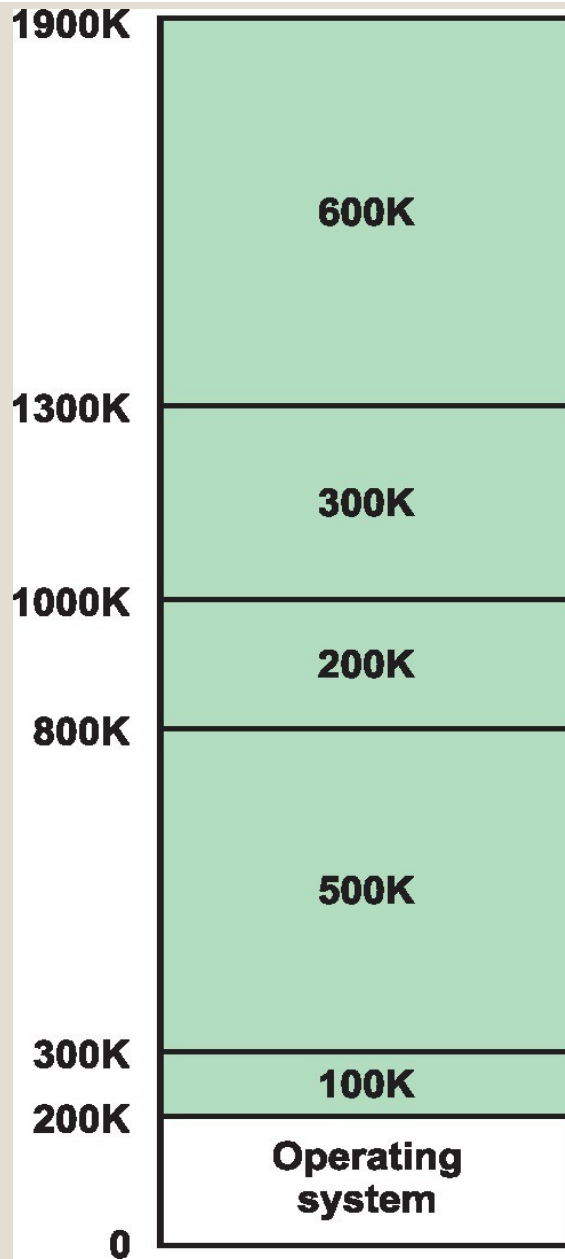


- **Example 2:** Given memory partition of 100K, 500K, 200K, 300K, and 600K in order. How would each of the first fit, best fit and worst fit algorithms place the processes of 212K, 417K, 112K and 426K in order? Which algorithm makes the most efficient use of memory? Show the diagram of memory status in each case.

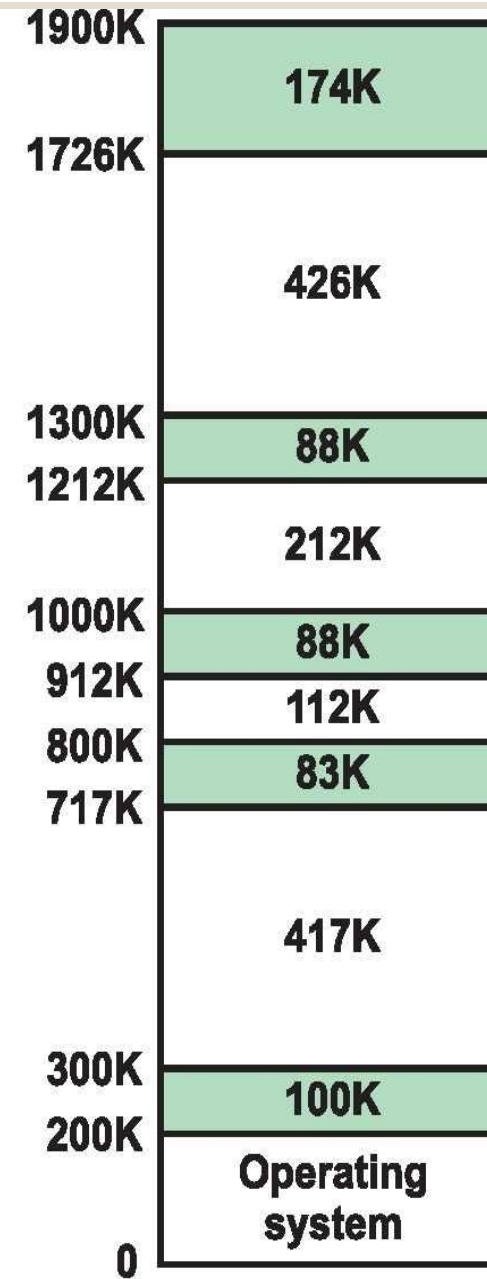
Solution: Assuming that the operating system resides in lower part of main memory and occupies 200K, the initial status of memory is shown



- Place the processes of 212K, 417K, 112K and 426K
- Best Fit
- First Fit
- Worst Fit



(a) First-fit



(b) Best-fit



(c) Worst-fit

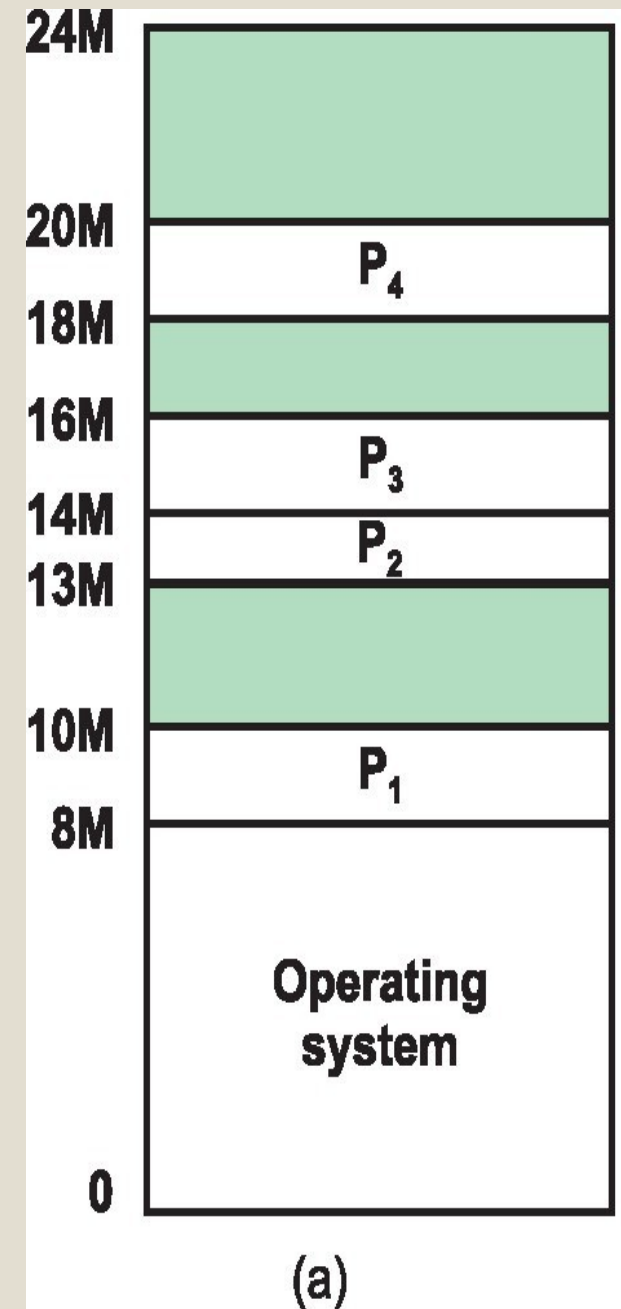
Fragmentation Problem

- An important issue related to contiguous multiple partition allocation scheme is to deal with **memory fragmentation**.
- There are two facets to memory fragmentation: **internal and external fragmentation**.
- **Internal fragmentation** exists in the case of memory having multiple fixed **partitions** when the **memory allocated** to a process is **not fully used by the process**.
- **External fragmentation** (also known as **checker boarding**) occurs in the case of memory having multiple variable partitions **when the total free memory in the system is large enough to accommodate a waiting process but it cannot be utilized as it is not contiguous**.

Fragmentation Problem

◦To understand the external fragmentation problem, consider the memory system (map along with free storage list) shown in Figure.

◦Now, **if a request for a partition of size 5M arrives**, it cannot be granted because **no single partition is available** that is large enough to satisfy the request. However, the **combined free space is sufficient** to satisfy the request.



Solution

- To get rid of external fragmentation problem, it is desirable to **relocate (or shuffle) some or all portions of the memory in order to place all the free holes together at one end of memory to make one large hole.**

- This technique of reforming the storage is termed as **compaction.**

- Compaction results in the **memory partitioned into two contiguous blocks—one of used memory and another of free memory.** Figure shows the memory map after performing compaction.

- Compaction may take place at the moment **any node frees some memory or when a request for allocating memory fails, provided the combined free space is enough to satisfy the request.**

- Since it is **expensive in terms of CPU time,** it is rarely used.

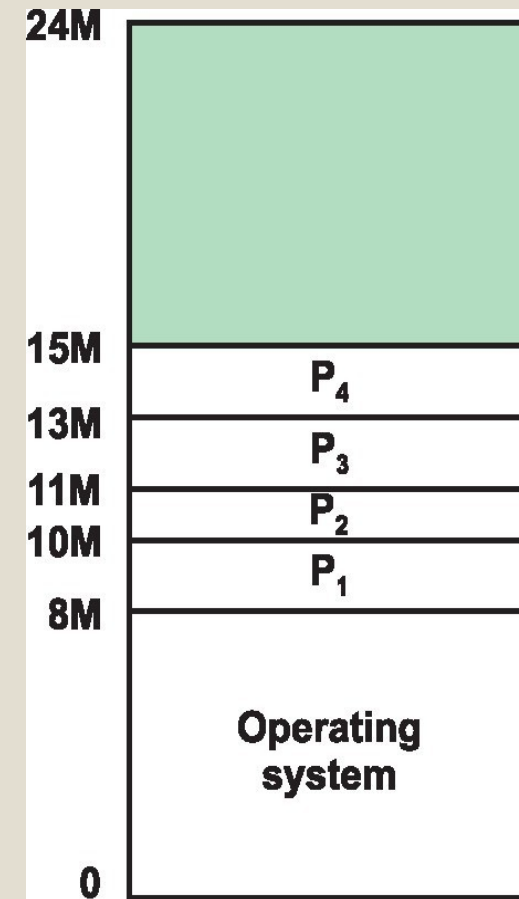


Fig. 7.9 Memory after Compaction

Difference Between Internal and External Fragmentation

Internal Fragmentation	External Fragmentation
Wasted memory inside an allocated memory block.	Wasted memory between allocated memory blocks.
Occurs when a process is allocated more memory than it actually needs .	Occurs when enough total free memory exists, but it is scattered into small non-contiguous blocks .
Common in fixed-size memory allocation (Fixed Partitioning, Paging).	Common in variable-size memory allocation (Dynamic Partitioning, Segmentation).
The unused memory cannot be allocated to another process.	Free memory exists but cannot be used because it is not contiguous.
Compaction does not solve internal fragmentation.	Compaction can reduce or eliminate external fragmentation.
Memory is wasted within the allocated partition.	Memory is wasted outside the allocated partitions.